

Shouyihuo

An inspectable 3D training simulation

A local-first, bilingual browser prototype connecting observation, diagnosis, simulated treatment and verification to one explicit state model. English is the default.

3 fault cases

3 learning modes

No backend or API key

Shouyihuo | 3D training lab | Course | **Workspace** | Learning records | English | 中文 | Local / v0.2

Independent assessment / Scenario 03

← **No refill after draining** → Start again | Finish & view report →

01 Observe | 02 Diagnose | 03 Repair | 04 Retest | Observe the water level, supply and component behaviour. Record at least 2 applicable observations before submitting a diagnosis.

Your task
The water level remains low after draining, with no visible refill. Check the supply before deciding what action to take.

Components 09

- 01 Cistern shell
- 02 Cistern lid
- 03 Supply valve
- 04 Supply pipe
- 05 Inlet assembly**
- 06 Float and lever
- 07 Flush assembly & seal
- 08 Overflow pipe
- 09 Water and level

Overview | Top view | Focus selection | Cutaway | Exploded view | Labels

3D WORKSPACE | Front wall hidden · interior visible

Drag to orbit · scroll to zoom | Open lid >

Supply: **Supply off** | Approx. water level: **Below the normal working level** | Flow observation: **No continuous flow**

Action log | 1 entry >

Observe & diagnose | Simulated repair

Scroll for diagnosis, repair and feedback

Selected component: **05 Inlet assembly**

Record observation

Check water level | Check supply

Works with the float to control filling.
Model notes >

Choose a diagnosis | Use your evidence

Choose the cause that fits the evidence >

Before removal: supply off → confirm isolation → drain → open lid. Observation and checks need no isolation.

Working-state check: **Not yet tested**

Run retest

Not yet retested

Teaching prototype. Not reviewed by a repair professional; applies only to the built-in simplified model. | Progress saved automatically · Water level and flow are teaching approximations

Actual refined WebGL workbench, recorded on 5 October 2026. English is the default; the English / Chinese switch preserves the attempt and its assessment.

01 The problem: keep visual actions and rules consistent

A convincing model is insufficient if an unsafe action silently succeeds or a camera change affects the score. The prototype makes these constraints testable: rejected disassembly preserves the assembly, records the error and prevents a later safety pass.

Built-in course: a simplified gravity-fed toilet cistern with a float-controlled inlet and flapper-style drain seal. Cases cover seal failure, inlet-control failure and a closed supply valve; the last requires no replacement.

Scope: an unaudited teaching model, not repair guidance, a calibrated digital twin or a professional qualification.

One state. Several views.

Typed transitions connect the lesson, 3D scene, assessment and local records.

Typed user events

Observe, diagnose, remove, replace, retest, tick



Pure simulation reducer

Guards + water approximation + verification + score



Derived interface and validated persistence

React / Three.js read state; localStorage restores it

02 Determinism with bounded claims

The same initial state and event sequence produce the same outcome. The UI advances the model by 0.25 simulation seconds per tick; the reducer uses internal steps of at most 0.05 seconds. Water is normalised teaching state, not a measured hydraulic quantity.

03 Safety is a gate, not a score deduction

Evidence, diagnosis, process and verification receive 25, 30, 25 and 20 points. Passing also requires correct diagnosis and treatment, complete verification and no critical safety error. Even a 100-point attempt may fail.

04 A live simulation, an on-demand renderer

An explicit visual projection isolates geometry from elapsed-time and log updates. Camera, water and part transitions request frames until settled; active flow keeps drawing. Local procedural lighting separates materials without network assets. These changes never dispatch repair events or award points.

05 Persistence preserves both progress and errors

Schema-validated localStorage keeps separate mode sessions and archives each completed attempt once by ID. The original attempt key and schema remain compatible. Locale is a separate preference; changing language cannot repair a fault or erase an error.

Inspect in the repository: `src/domain/engine.ts`, `src/storage/local.ts`, `src/components/TankModel.tsx`, `tests/engine.test.ts` and `e2e/flows.spec.ts`.

Evidence with clear limits

Review the behaviour, the implementation and the recorded checks together.

06 Verification record

The **5 October 2026 refinement run** passed type checking, **56 unit tests**, **21 browser tests** and a production build. Logs, software-WebGL checks and fresh desktop/mobile captures are in **docs/portfolio/VALIDATION.md**. Earlier results remain dated historical evidence.

Nine invariant tests exercise 3,888 seeded state transitions and 1,944 persisted continuation steps. Browser tests cover all three cases, real mesh selection, safety rejection, old records, language changes, idle-render wake-up and live context loss.

07 A failure that changed the implementation

Demand rendering exposed a label-cache bug: a stationary cutaway change could retain stale occlusion. The correction includes geometry changes in cache invalidation and updates world matrices before raycasting. The browser regression toggles the wall and checks actual label visibility.

In one matched idle scene, actual WebGL draw calls changed from 5,180 over approximately 3 seconds to zero. The simulation clock continued advancing. This demonstrates removed idle drawing; it does not measure frame rate, energy use or native-GPU performance.

08 Review in five minutes

Run. Use `npm ci` and `npm run dev` from the project folder. Open `http://127.0.0.1:5173/review/` for the English-default bilingual introduction.

Interact. Follow `docs/portfolio/REVIEWER_GUIDE.md` to complete the supply-valve case, inspect a score report and refresh the saved record.

Challenge. Start a new attempt, request premature removal and inspect the rejected action. Complete the case to see why a numeric score alone cannot pass.

09 Contribution and limitations

AI-assisted development: the project owner supplied the product brief and acceptance constraints. OpenAI Codex contributed substantial implementation, testing, debugging and documentation. The artefact does not imply unaided solo coding; personal contribution claims require their own evidence.

No professional repair review, learner study, physical validation or improved-learning claim is made. Native-GPU performance, every browser/device and physical printing have not been established. Local records are editable and are not formal certificates. The project demonstrates software design and verification practices, not machine learning or computational fluid dynamics.

Reading order: `README.md`, `docs/RENDERING_REFINEMENT.md`, the case study and validation report in `docs/portfolio/`, then source and tests. Contribution details: `docs/portfolio/PROVENANCE.md`.